

Module 14 — Connect Your Agent to the Places It Works

Seven technical how-to scripts · no protagonist · interfaces, diagrams and terminal only

Rewritten 2026-09-16 against `notion-slack-railway-github-guide.md`, grounded in the team's own ILHAM system — so every step and code shape is real, not hypothetical.

The premise: you stop opening the project and chatting with it. You ask in Slack. It works in the background. It tracks itself in Notion. It comes back when it is done, or stuck.

Videos 3–6 are walkthroughs on the real interfaces, each with the exact screens to capture.

Deployment is platform-agnostic throughout — Railway, Render, Vercel or Fly; the module never tells anyone to set up a new host.

Every video pauses for a question the learner must answer to continue.

No art, voiceover or render has been generated. Nothing is built until you approve the scripts.

Contents

- 01 Module overview — the seven videos, and the format change
- 02 Video 1 · Why a Chat Window Is Not Enough (§1)
- 03 Video 2 · The Loop, End to End (§2)
- 04 Video 3 · Notion — The Task Board (§3)
- 05 Video 4 · Slack — Speak, Then Listen (§4a + §4b)
- 06 Video 5 · GitHub — Two Jobs (§5)
- 07 Video 6 · Your Deployment — The Body (§6)
- 08 Video 7 · The Four Mistakes (§7)
- 09 Appendix · Research brief (every fact, sourced)

Module 14 — Connect Your Agent to the Places It Works

Format changed 2026-09-16: no protagonist, no storyline. These are **technical how-to explainers** — interface screens, diagrams and terminal output, narrated directly to the person building it. Ali does not appear. Nothing is illustrated.

Content is taken section by section from `notion-slack-railway-github-guide.md`, the standalone reference grounded in the team's own ILHAM system (`Documents/Intelligence project/app/`) — so every step, value name and code shape is real, not hypothetical.

The premise the whole module builds toward

You stop opening the project and chatting with it. You ask in Slack. It works in the background. It tracks itself in Notion. It comes back to you when it's done or stuck.

Any agentic system you want genuinely autonomous needs three things a chat window cannot give it:

1. **A place to keep track of what it's doing** — a task board it can read and write 2. **A way to talk to you, in both directions** — you tell it what to do; it tells you what happened 3. **A body that's awake when you are not** — something running whether or not your laptop is open

Notion, Slack, GitHub and your deployment are the four pieces that answer those three needs.

Deployment is platform-agnostic. Learners already have something live — Railway, Render, Vercel, Fly. The module never tells anyone to set up a new host; video 6 is what to add *on top of what you already run*. Railway appears only as the reference system's example.

The seven videos

#	Title	Covers	The one move
1	Why a Chat Window Isn't Enough	§1	Name the three things your setup is missing
2	The Loop, End to End	§2	Trace one request from Slack message to merged PR
3	Notion — The Task Board	§3	Create it, build the board, share it under Connections , copy two values
4	Slack — Speak, Then Listen	§4a + §4b	Scopes → install → <code>xoxb-</code> → <code>/invite</code> → channel ID, then Event Subscriptions → Request URL → challenge → signing secret
5	GitHub — Two Jobs	§5	A token to commit with, and a webhook to be woken by
6	Your Deployment — The Body	§6	Paste the secrets, give Slack and GitHub your address, add the check-in loop
7	The Four Mistakes	§7	Identify which of four before touching any code

Order is build order. 1–2 give the shape; 3 → 4 → 5 → 6 build it in the order the values are needed (you cannot set Slack's Request URL before \$6 gives you an address — video 4 says so and video 6 pays it off); 7 is the troubleshooting pass you come back to.

What this format means in practice

Visually: every beat is one of three things — a **real interface screen** (ui), an **HTML card or diagram** (info), or a **full-screen title/signpost** (card). No characters, no shop, no scenes.

Practically, it is cheaper and faster than the illustrated format:

	Illustrated (old)	Technical how-to (this)
Imagen art	~15 images/video	none
Cutout segmentation	required	none
kie i2v clips	3/video, paid	none
Cost per video	~\$0.60 art + \$0.90 motion	voiceover only, pennies
Gates	qa-visuals character rules	qa-info + qa-checkpoint only

qa-visuals detects the absence of character art and skips its character rules automatically — it does not need to be told which format a video is.

The interfaces are shown exactly. Videos 3–6 each carry a screen-by-screen capture list: what must be visible, and what must never be (real tokens, workspace names, colleagues, customer data). Two production routes — faithful HTML re-creations in animation/walkthrough.html , or real captures with a scrub pass. Re-creation is recommended: proofread-able, re-renders when a vendor moves a button, and cannot leak.

The question, in every video (settled — enforced, not remembered)

Each video **pauses** at one checkpoint. The LMS shows a multiple-choice question. **The learner must click an option to continue** — no skip, no dismiss, no seeking past it. A wrong answer gets feedback that explains the mistake; a right one gets confirmation of why. Then playback resumes.

Nothing about the question is drawn or spoken, so it is invisible in the MP4 — which is why qa-checkpoint.js fails any build whose checkpoint is missing, drawn, spoken, first/last, un-answerable, or whose wrong-answer feedback is under 120 characters or a single sentence. scripts/publish-checkpoint.js then sends the timing and the question to Railway and **verifies it live** before reporting success, so the LMS developer can wire it up.

What this supersedes

Scripts/_superseded/ holds the original eight scripts (the "Ali connects two tools" framing) and the first rewrite (same content, Ali retained). Three built assets are now out of date and would need rebuilding to match: the module intro, Locked Out of the Room, and the assessment guide — all three illustrated, all three story-led. Their checkpoints are live on Railway and would need republishing.

Video 1 — Why a Chat Window Isn't Enough — draft

Format: technical how-to. No protagonist. Diagrams and screens only.

Covers: guide §1

Teaches: an agent you have to open and talk to is a chat window with extra steps; three specific capabilities are missing, and four services supply them.

One move: name which of the three your setup is missing today.

Runtime: ~2 min · no art, no cutouts, no i2v — HTML and voice only.

Teaches no setup. Its job is to replace "I chat with my agent" with "I ask, and it works without me", so videos 3–6 feel like four jobs rather than four logos.

Script (one beat per line · [card] = full-screen card · [info] = diagram/data card · [ui] = real screen)

1	CARD	If you have to open it and talk to it, it is a chat window with extra steps. <i>(title)</i>
2	UI	This is the normal setup: an editor open, a prompt typed, an answer coming back. <i>(U1)</i>
3	UI	It works. It works well. And it only works while you are sitting there. <i>(U1 held)</i>
4	INFO	Close the laptop and the agent stops existing. <i>(statement)</i>
5	INFO	That is not an agent doing your work. That is you, doing your work, faster.
6	INFO	Three capabilities are missing, and none of them is intelligence. <i>(threepart, empty)</i>
7	INFO	One — somewhere to keep track of what it is doing. <i>(threepart, panel one)</i>
8	INFO	Not in its context window, and not in a chat history that scrolls away. <i>(panel one detail)</i>
9	INFO	Two — a way to talk to you, in both directions. <i>(panel two)</i>
10	INFO	You tell it what to do. It tells you what happened, or that it is blocked. <i>(panel two detail)</i>
11	INFO	Three — a process that is running when you are not there. <i>(panel three)</i>
12	INFO	Four services supply those three. <i>(fourparts — Notion · Slack · GitHub · your deployment)</i>
13	INFO	Notion is the task board: what it is doing, and what it did. <i>(fourparts, panel one lit)</i>
14	INFO	Slack is the two-way line: your instructions in, its reports out. <i>(panel two lit)</i>
15	INFO	GitHub is where the work lands, as commits and pull requests you can read. <i>(panel three lit)</i>
16	INFO	Your deployment is the process that stays up. <i>(panel four lit)</i>
17	INFO	You already have that last one. It is whatever your project is deployed on today.
18	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
19	INFO	None of the four is the point. The loop between them is the point. <i>(statement)</i>
20	UI	Here is the whole thing in one sentence. <i>(U2 — the premise on screen)</i>
21	INFO	You ask in Slack. It works in the background. It tracks itself in Notion.
22	INFO	And it comes back to you when it is done, or when it is stuck.

The checkpoint (beat 18)

Stem. Your agent answers correctly every time you open the project and type a prompt. Which of the three capabilities does it have?

Options.

- A — Somewhere to track its work
- B — A two-way line to you
- C — A process running when you are not there
- D — None of them

Answer. D.

If they get it right. Correct — and that is why "just connect one thing" never feels like enough. When the only way to reach it is to open the project, the chat scrollbar *is* its tracking, your typing *is* the entire conversation, and your laptop *is* the process. All three jobs are being done by you being present. Each needs handing to something that outlives the session.

If they get it wrong. It has none of them, which surprises most people because the setup clearly works. Look at what happens when you walk away: the task history disappears with the session, so there is no tracking; nothing can reach you — and you cannot reach it — without you opening the editor, so there is no two-way line; and the process itself is your laptop, so there is nothing running at all. Answering well while you watch is not the same as having any of the three. That is the gap the next six videos close.

Screens

ID	Screen	Must be visible	Must NOT be visible
U1	An editor with an agent session	A typed prompt, an answer streaming back, nothing else running	Real project code, real file names
U2	The premise, as a full-width card	The four sentences, one per line, set large	—

Gate check

READY · 23 beats · 2 [ui] · 18 [info] · 2 [card] · 1 [checkpoint] · ~2:00

- **qa-visuals** **skips this automatically** — no **ali** or **scene** beats, so the character rules do not apply. **qa-info** and **qa-checkpoint** are the gates that matter.
- **Checkpoint:** beat 18, between two spoken beats (17 → 19), 4 options, both halves authored; the wrong-answer feedback works through all three capabilities rather than just naming the answer.
- **Data templates:** threepart built across 6–11, fourparts built across 12–16, statement ×2 (4, 19). Two distinct diagrams plus two statement cards — at the statement limit, which is fine here because the diagrams carry the teaching.
- **Covers \$1 completely:** the three needs (7–11), the four services mapped to them (12–16), the "you already have the fourth" point (17), and the premise verbatim (20–22).

Build notes

- **Beats 6–11 are ONE diagram built in three passes**, and **beats 12–16 are ONE diagram lit panel by panel**. Not eleven cards. The whole video is two diagrams and a screen.
- **Beats 21–22 are the module thesis** — unhurried, with air between the sentences. Quoted again in the assessment.
- The fourparts card recurs in every later video as a "you are here" marker; this is where its geometry is set.
- No Imagen, no segmentation, no kie. Build is: qa-info → qa-checkpoint → TTS → compile → stitch → verify.

Video 2 — The Loop, End to End — draft

Format: technical how-to. No protagonist. One animated diagram, plus the real screens it describes.

Covers: guide §2 — the loop diagram and the four role definitions

Teaches: one request follows a fixed path — Slack in, Notion card, GitHub pull request, Slack out — and the whole circuit runs on a process that polls every 60 seconds and reacts to webhooks instantly.

One move: trace your own next request around the four before building any of them.

Runtime: ~2 min · HTML and voice only.

This is the map videos 3–6 fill in. The learner should finish able to say where a request goes and what each service is responsible for.

Script (one beat per line · [card] = full-screen card · [info] = diagram/data card · [ui] = real screen)

1	CARD	One request. Four services. A path it always takes. <i>(title)</i>
2	UI	It starts with a message: "add a login page". <i>(L1 — Slack composer)</i>
3	INFO	Slack is the conversation — the only place you touch. <i>(loop diagram, Slack lit)</i>
4	INFO	Everything after this happens without you. <i>(loop diagram, arrow moving)</i>
5	UI	A card appears on the board: Add login page, status To Do. <i>(L2 — Notion board)</i>
6	INFO	Notion is the operating area — where the work is tracked. <i>(loop, Notion lit)</i>
7	INFO	Not in the agent's context, and not in a chat that scrolls away.
8	UI	The agent picks the top card and moves it to In Progress. <i>(L3 — status changing)</i>
9	UI	It writes the code, commits, and opens a pull request. <i>(L4 — the PR)</i>
10	INFO	GitHub is the workshop — where the work actually lands. <i>(loop, GitHub lit)</i>
11	INFO	A change you can read, question, approve, or refuse.
12	UI	The card moves to Done, and a message arrives back in the channel. <i>(L5 — "login page is up, PR #42")</i>
13	INFO	That is the loop. You touched it once. <i>(loop diagram, complete)</i>
14	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
15	INFO	One thing is missing from that diagram, and it is the one nobody draws. <i>(loop, dimmed)</i>
16	INFO	Something has to be running to do any of it. <i>(loop, the host layer appearing underneath)</i>
17	INFO	A small server that never turns off. <i>(host layer detail)</i>
18	INFO	It checks for new work every sixty seconds. <i>(twocard — the timer)</i>
19	INFO	And reacts to a GitHub webhook the instant it fires. <i>(twocard — the trigger)</i>
20	INFO	Polling finds work. Webhooks find it faster. You want both.
21	INFO	Ask · track · build · report — on something that stays awake. <i>(loop, complete with host)</i>

The checkpoint (beat 14)

Stem. Your agent is connected to Slack, Notion and GitHub, and the loop works exactly as shown. You shut your laptop and go home. What happens to the next request someone sends?

Options.

- A — It queues and runs when you open the laptop again
- B — Nothing happens; there is no process running to receive it
- C — Slack retries until the agent responds
- D — Notion creates the card and the rest waits

Answer. B.

If they get it right. Exactly. Slack, Notion and GitHub are services the agent *calls* — none of them runs it. The code doing the reading, deciding and committing has to be executing somewhere, and if that somewhere is your laptop, closing it ends the loop.

If they get it wrong. Nothing happens at all, and the reason it is tempting to think otherwise is that these services feel active. They are not. Slack delivers an event to a URL; if nothing answers, it retries briefly and gives up — it does not hold messages for an app that is switched off. Notion only creates a card when something calls its API, and nothing is calling. Your agent is a process, and a process that is not running cannot poll, cannot be notified, and cannot create anything. Every connection you build points at a thing that is off until the agent lives somewhere that stays awake.

Screens

ID	Screen	Must be visible	Must NOT be visible
L1	Slack composer	The typed request in a channel	Real colleagues, real channel names
L2	Notion board	One new row: Name, Status = To Do, Priority, Notes	Real tasks
L3	Same board, status changed	Status = In Progress, Notes line added by the agent	—
L4	GitHub pull request	Title, author = the agent, Files changed tab	Real code
L5	Slack channel	The agent's reply with a PR reference	Real message history

Gate check

READY · 22 beats · 5 [ui] · 14 [info] · 2 [card] · 1 [checkpoint] · ~2:00

- **qa-visuals** **skips this automatically** (no character art). **qa-info** + **qa-checkpoint** are the gates.
- **Checkpoint:** beat 14, between two spoken beats (13 → 15), 4 options, both halves authored; the wrong-answer feedback refutes each wrong model specifically rather than restating the right one.
- **Covers §2 completely:** the full path (2–12), all four role definitions in the guide's own words — operating area, conversation, workshop, body (3, 6, 10, 16) — the 60-second poll (18) and the webhook trigger (19).

- **Data templates:** the loop diagram (used 3, 4, 6, 10, 13, 15, 16, 21 — one evolving object), twocard (18–19). Two distinct; zero plain statement cards.

Build notes

- **The loop diagram is the video.** One object, built and lit across eight beats, with the host layer appearing *underneath* it at beat 16 — that reveal is the whole point of the second half, so give it the strongest transition in the file.
- Beats 18–19 are **one card with two states** (timer, then trigger), not two cards.
- The five real screens (L1–L5) are the same five the learner will see in videos 3–6, so use identical invented data — same task name, same PR number — and the module reads as one continuous example.
- No Imagen, no segmentation, no kie. Build is: qa-info → qa-checkpoint → TTS → compile → stitch → verify.

Video 3 — Notion, The Operating Area — draft

Format: technical how-to walkthrough. No protagonist. Real Notion screens, **card** + **ui** beats, cursor visible, no illustrated art.

Covers: guide §3, every step

Teaches: the agent gets a task board it can read and write, which means an integration, a database with four columns, the sharing step everybody misses, and exactly two values handed over.

One move: create it, build the board, **share it under Connections**, copy the two values.

Spine: by the end of this video a card the agent created appears on your board while you watch.

Built for following, not watching: one action per beat, every field named before it is typed. Nothing conceptual — videos 1 and 2 earned the ideas.

Deferred: Slack (video 4), GitHub (video 5), where the values are pasted (video 6).

Script (one beat per line · [card] = full-screen card · [ui] = interface screen · [info] = spoken point on a card)

1	CARD	Five steps, and the agent has a board it can read and write. <i>(title card)</i>
2	INFO	We are building the place your agent keeps track of its own work.
3	UI	Go to notion dot so slash my dash integrations, and click New integration. <i>(N1)</i>
4	UI	Name it after the agent, pick the workspace, and submit. <i>(N2)</i>
5	UI	Copy the Internal Integration Secret. This is the agent's key — treat it like a password. <i>(N3)</i>
6	CARD	One down. The next two are the board itself. <i>(signpost)</i>
7	UI	Make the database that will be the task board. <i>(N4)</i>
8	UI	Four columns, and the names matter: Name, Status, Priority, Notes. <i>(N5)</i>
9	UI	Status is a select — to do, in progress, done, blocked. <i>(N6)</i>
10	UI	Priority is a select too — low, medium, high — so it knows what to pick up first. <i>(N6b)</i>
11	UI	Notes is where the agent writes its own log as it works. <i>(N7)</i>
12	CARD	Now the step everybody misses. <i>(signpost — hold)</i>
13	UI	Open the board, click the three dots, top right. <i>(N8)</i>
14	UI	Choose Connections, and add the integration by name. <i>(N9)</i>
15	UI	The key alone does nothing. Notion also needs you to share the page with it. <i>(N9 held)</i>
16	INFO	Same as inviting a person. Having a key is not the same as being let in.
17	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
18	UI	Last, the database ID — the thirty-two characters in the URL, before the question mark. <i>(N10)</i>
19	UI	The part after v equals is the view, not the board. <i>(N10, second highlight)</i>
20	CARD	Two values go to the agent: the secret, and the database ID. <i>(checklist)</i>

- 21 UI Test it by having the agent create one card. (N11 — code, then the card appearing)
- 22 UI If it appears on your board, the connection is real. (N12 — the board with the new row)
- 23 INFO That card was not typed by a person. That is the whole point.
- 24 INFO Your turn. Build the board, share it, and make one card appear.

The checkpoint (beat 17)

Stem. You created the integration, copied the secret, and your code returns a 404 saying the database does not exist — but you are looking straight at it in your browser. What is wrong?

Options.

- A — The secret was copied incorrectly
- B — The database was never shared with the integration under Connections
- C — The database ID includes the view ID
- D — Notion needs a paid plan for API access

Answer. B.

If they get it right. Exactly — and note how the error lies to you. It says "does not exist", which sends people hunting for a typo in the ID, when the real meaning is "exists, but not for you". Creating an integration grants it nothing until you open the page, choose Connections, and add it by name.

If they get it wrong. It is the sharing step, and the reason it catches everyone is that you can see the database perfectly well — but you are signed in as *you*, and the integration is not you. It has its own identity with access to nothing until a human explicitly shares a page with it. A mistyped secret would fail as an authentication error, not a missing object; a view ID in the database ID gives a different, clearer error; and none of this requires a paid plan. Whenever Notion says something does not exist while you are staring at it, check Connections first.

Screens to capture or re-create

ID	Screen	Must be visible	Must NOT be visible
N1	<code>notion.so/my-integrations</code>	The New integration button	Other integrations, the real workspace name
N2	New integration form	Name field filled with the agent's name, workspace picker, Submit	The real workspace name
N3	Integration → Internal Integration Secret	The Show/Copy control, the secret masked as dots, the <code>ntn_ / secret_</code> prefix only	Any real secret characters
N4	A new Notion database	The empty board/table view	Real page titles in the sidebar
N5	The four columns	<code>Name</code> (title), <code>Status</code> , <code>Priority</code> , <code>Notes</code> — added one at a time	—
N6	<code>Status</code> select options	To Do · In Progress · Done · Blocked	—
N6b	<code>Priority</code> select options	Low · Medium · High	—

ID	Screen	Must be visible	Must NOT be visible
N7	Notes rich-text column	An invented agent log line	Real project data
N8	The board → ... menu open	The three-dot button, the open menu	Real page titles
N9	Connections → search → confirm	The typed integration name, the result row, the confirmation about access	—
N10	The board's URL in the address bar	The 32-character ID highlighted; the ?v=... view ID highlighted separately, in a different colour	The real workspace subdomain
N11	Editor/terminal: create-or-update	The query-then-create shape; NOTION_API_KEY and NOTION_GOALS_DB_ID as names only	Any real key
N12	The board with the agent's card	One new row: Name, Status = In Progress, a Notes line	Real customer or task data

Production: re-create in `animation/walkthrough.html` (recommended — proofread-able, re-renders when a vendor moves a button, cannot leak) or capture real screens with a scrub pass for names and tokens.

Gate check

READY · 24 beats · 15 [ui] · 4 [card] · 4 [info] · 1 [checkpoint] · ~2:15

- **skips this automatically** — no character art, so its §3d rules do not apply. The bar here: one action per screen, a visible cursor, every element named before it is clicked.
- **Checkpoint:** beat 17, between two spoken beats (16 → 18), 4 options, both halves authored, wrong-answer feedback explains why the error message misleads. Passes `qa-checkpoint`.
- **Covers §3 completely:** integration (3–5), the four columns with their options (7–11), Connections (12–15), database ID and the view-ID trap (18–19), the two values (20), the create-or-update test (21–22).
- **No assignments.** Beat 23 is a spoken instruction to do the thing they just watched.

Build notes

- **No Imagen art, no i2v.** Motion is cursor travel, click pulses, typed text, menus opening.
- **Step counter** in the corner (`3 / 5`), advancing on beats 3, 7, 13, 18, 21.
- **Beats 12–15 are the heart of the video** — one continuous browser frame, no cuts, the cursor moving from the three dots to Connections to the confirmation. That path is what people skip.
- Beat 19 must **contrast both IDs in one frame**, two highlight colours, the wrong one marked.
- Beat 21's code is shown as four or five lines maximum — the query-then-create shape, not the full file.
- The spoken-point beats are plain typographic cards — one sentence, set large, no illustration.
- Every masked value is masked **in the render**, never blurred afterwards.

Video 4 — Slack, Both Directions — draft

Format: technical how-to walkthrough. No protagonist. Real Slack screens, **card** + **ui** beats, cursor visible.

Covers: guide §4a **and** §4b — merged, because speaking and listening are one app configured in one sitting, and splitting them leaves a learner holding a half-built Slack app across two videos.

Teaches: outbound first (scopes, install, **xoxb-**, **/invite**, channel ID), then inbound (Event Subscriptions, the Request URL, the challenge echo, the signing secret) — and why the second half is what makes it two-way.

One move: get four values and put the bot in the channel.

Spine: the agent posts its first message, then answers when you @-mention it.

The longest video in the module (~4 min), and the only one with a deliberate hand-off: the Request URL cannot be filled in until video 6 gives them a public address. The script says so out loud at beat 19 rather than letting anyone sit stuck on a form they cannot complete yet.

Script (one beat per line · [card] = full-screen card · [ui] = interface screen · [info] = spoken point on a card)

1	CARD	Two halves: let it speak, then let it listen. <i>(title card)</i>
2	INFO	Most people do the first half and stop, and then wonder why it never replies.
3	CARD	Half one — outbound. <i>(signpost)</i>
4	UI	Go to a p i dot slack dot com slash apps, and click Create New App. <i>(S1)</i>
5	UI	From scratch, name it, pick the workspace. <i>(S2)</i>
6	UI	Open OAuth and Permissions, and scroll to Bot Token Scopes. <i>(S3)</i>
7	UI	Add chat colon write — that is permission to send messages. <i>(S4)</i>
8	UI	Add app underscore mentions colon read — so it sees when it is @-mentioned. <i>(S5)</i>
9	UI	Add channels colon history — so it can read the channel it is watching. <i>(S6)</i>
10	UI	Scroll up, Install to Workspace, and approve. <i>(S7)</i>
11	UI	Copy the Bot User OAuth Token — the one starting x o x b. <i>(S8)</i>
12	CARD	The step everybody misses, again. <i>(signpost — hold)</i>
13	UI	In Slack itself, go to the channel and type slash invite, then your bot's name. <i>(S9)</i>
14	UI	A token does not put the bot in a channel. Somebody has to invite it. <i>(S9 held)</i>
15	INFO	Same rule as Notion's Connections. The credential is necessary. It is not sufficient.
16	UI	Right-click the channel, view channel details, copy the Channel ID from the bottom. <i>(S10)</i>
17	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
18	CARD	Half two — inbound. This is what makes it two-way. <i>(signpost)</i>
19	UI	Event Subscriptions, toggle on — and here you need something we do not have yet. <i>(S11)</i>

- 20 **INFO** The Request URL has to be a real, public address that is already running your code.
- 21 **INFO** That comes in video six. Leave this tab open and come back to it.
- 22 **UI** Under Subscribe to bot events, add app underscore mention. (S12)
- 23 **UI** And message dot channels, so it reacts to any message in a channel it is in. (S13)
- 24 **UI** The moment you paste the URL, Slack sends a one-time challenge. (S14)
- 25 **UI** Your server has to echo that challenge value back, or the URL never verifies. (S15 — the code)
- 26 **UI** Last value: Basic Information, copy the Signing Secret. (S16)
- 27 **UI** That secret is how your server proves a request really came from Slack. (S17 — the HMAC check)
- 28 **INFO** Anyone can POST to a public URL. The signature is what makes it trustworthy.
- 29 **CARD** Four values: bot token, signing secret, channel ID — and the bot, invited. (checklist)
- 30 **UI** Outbound is one call: chat dot postMessage with your token and channel. (S18)
- 31 **INFO** Speak, then listen. Do both, and you have a colleague instead of a printer.
- 32 **INFO** Your turn. Get the three values and invite the bot; the URL waits for video six.

The checkpoint (beat 17)

Stem. Your bot has `chat:write`, the app is installed, and the token is correct — but posting to your channel fails with `not_in_channel`. What is missing?

Options.

- A — The `channels:history` scope
- B — The bot was never invited to that channel
- C — The app needs reinstalling after adding scopes
- D — The channel ID is wrong

Answer. B.

If they get it right. Exactly, and the error name says it plainly once you read it as a statement of fact rather than a failure: the bot is *not in the channel*. Scopes say what it is allowed to do anywhere; membership says where it is allowed to do it. `/invite @your-bot` is the whole fix.

If they get it wrong. The bot is not a member of the channel. This is the single most common Slack error, and it catches people because everything else is genuinely correct — the token works, the scope is granted, the app is installed. But a scope is a permission, not a presence: `chat:write` means "may send messages", not "is in this room". `channels:history` governs reading, not posting. Reinstalling matters when you *add* a scope, which is a different error (`missing_scope`). And a wrong channel ID gives `channel_not_found`, not `not_in_channel` — the error is telling you it found the channel fine and your bot simply is not in it.

Screens to capture or re-create

ID	Screen	Must be visible	Must NOT be visible
S1	<code>api.slack.com/apps</code>	The Create New App button	Other apps, the real workspace name

ID	Screen	Must be visible	Must NOT be visible
S2	From scratch modal	App name filled, workspace picker	The real workspace name
S3	OAuth & Permissions → Bot Token Scopes	The scope section, the Add button	Any user-token scopes — do not show that section
S4–S6	Each scope added in turn	<code>chat:write</code> , then <code>app_mentions:read</code> , then <code>channels:history</code>	—
S7	Install to Workspace → consent	What the app will be able to do, the Allow button	The real workspace name and icon
S8	Bot User OAuth Token	<code>xoxb-</code> prefix then dots, the Copy control	Any real token characters
S9	The channel composer	<code>/invite @your-bot</code> typed out, then the "added to the channel" line	Real colleagues' names or avatars
S10	Channel details, scrolled to the bottom	The Channel ID (<code>C0...</code>)	Real channel names in the sidebar
S11	Event Subscriptions toggled on	The Request URL box, empty, with its "we'll verify" note	—
S12–S13	Subscribe to bot events	<code>app_mention</code> , then <code>message.channels</code>	—
S14	The URL verification moment	Slack's one-time challenge request, the green Verified tick	A real public URL
S15	Editor: the challenge echo	The <code>url_verification</code> branch returning <code>challenge</code> — 3 lines	—
S16	Basic Information → Signing Secret	The Show/Copy control, value masked	Any real secret characters
S17	Editor: the signature check	<code>v0:timestamp:body</code> , HMAC-SHA256, <code>compare_digest</code> , 403 on mismatch	—
S18	Editor: sending a message	<code>chat.postMessage</code> with the token as a name, channel, text	Any real token

Production: re-create in `animation/walkthrough.html` (recommended) or capture real screens with a scrub pass. Never show a real workspace, channel or colleague.

Gate check

READY · 32 beats · 19 [ui] · 5 [card] · 7 [info] · 1 [checkpoint] · ~3:45

- **Longest in the module, deliberately.** Splitting it left the learner with a Slack app that could talk but not hear, across a video boundary — the merge is the fix, and beats 3 and 18 signpost the halves so it never feels like one undifferentiated slog.
- **The hand-off is explicit** (19–21): the Request URL needs video 6. Said out loud, with an instruction to leave the tab open, rather than leaving a learner stuck on a form.
- **Checkpoint:** beat 17, at the seam between the two halves, 4 options, both halves authored; the wrong-answer feedback distinguishes `not_in_channel` from `missing_scope` and `channel_not_found`, which is the actual confusion.

- **Covers §4a and §4b completely:** create (4–5), three scopes (6–9), install (10), token (11), invite (12–15), channel ID (16), Event Subscriptions (19), the two events (22–23), the challenge (24–25), the signing secret and HMAC (26–28), the outbound call (30).
- **The repeated pattern is named** at beat 15 (credential ≠ access, same as Notion) and again at 28 (a secret proves who is calling) — video 5 calls back to it.

Build notes

- **No Imagen art, no i2v.** Motion is cursor travel, click pulses, typed text, the verification tick.
- **Step counter** (4 / 9), advancing on 4, 6, 10, 11, 13, 19, 22, 26, 30.
- **Beats 12–15 run as one continuous Slack frame**, no cuts — the invite is the step that fails silently.
- Beats 25 and 27 show **three to five lines of code each**, no more. They are recognition aids, not listings; the same shape returns in video 5 for GitHub.
- Beat 24's challenge/verify moment should land as a single satisfying tick — it is the only moment in the module where a form tells you it worked.
- The spoken-point beats are plain typographic cards — one sentence, set large, no illustration.
- Every masked value is masked **in the render**.

Video 5 — GitHub's Two Jobs — draft

Format: technical how-to walkthrough. No protagonist. Real GitHub screens, **card** + **ui** beats.

Covers: guide \$5, both jobs

Teaches: GitHub is not "another app to connect" — it does two separate things. It is the agent's

workshop (a token lets it commit and open pull requests), and it is a **trigger** (a webhook wakes the agent the moment something happens, instead of it waiting for the clock).

One move: make a fine-grained token scoped to one repo, then add a webhook with a matching secret.

Spine: the agent opens a pull request, and later reacts to a repo event within seconds.

Keeping the two jobs separate is the whole point of the video — blurred together, "connect GitHub" sounds like one more API key, and the trigger half never gets built.

Script (one beat per line · [card] = full-screen card · [ui] = interface screen · [info] = spoken point on a card)

1	CARD	GitHub does two jobs. Most people build one and miss the other. <i>(title card)</i>
2	INFO	Job one is the workshop. Job two is the doorbell.
3	CARD	Job one — the workshop. <i>(signpost)</i>
4	UI	The agent's code lives in a repo, and it needs to commit like a person does. <i>(G1)</i>
5	UI	Settings, Developer settings, Personal access tokens, Fine-grained. <i>(G2)</i>
6	UI	Scope it to the one repository. Not all of them. <i>(G3)</i>
7	UI	Grant Contents, read and write. That is what lets it commit. <i>(G4)</i>
8	INFO	Fine-grained and one repo, so a leaked token cannot reach anything else.
9	UI	Copy the token. This is the credential that makes it a committer, not a reader. <i>(G5)</i>
10	UI	Now it can push commits and open pull requests on its own. <i>(G6 — a PR by the agent)</i>
11	INFO	A pull request is the agent proposing. You still decide.
12	CARD	Job two — the doorbell. <i>(signpost)</i>
13	INFO	So far it only acts when you message it, or when the clock says so.
14	UI	A webhook adds a third trigger: something happened in the repo. <i>(G7)</i>
15	UI	Repo, Settings, Webhooks, Add webhook. <i>(G8)</i>
16	UI	Payload URL is your deployed app's address plus the endpoint. <i>(G9)</i>
17	UI	Content type, application slash json. <i>(G10)</i>
18	UI	Secret: any string — and set the exact same string on your server. <i>(G11)</i>
19	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
20	UI	Pick the events to fire on. <i>(G12)</i>

- 21 **INFO** The real system listens for repository invitations, so it can accept being added to a repo on its own.
- 22 **UI** Your server checks the signature the same way it did for Slack. (*G13 — the HMAC check*)
- 23 **INFO** A webhook is just someone else's server calling yours.
- 24 **INFO** And the secret is how you prove it is really them.
- 25 **CARD** Two jobs, two credentials: a token to act, a secret to be trusted. (*checklist*)
- 26 **INFO** Your turn. Make the token, add the webhook, and set the secret in both places.

The checkpoint (beat 19)

Stem. You added the webhook with a secret, deployed your server, and pushed an event — but the server rejects every delivery with a 403. What is almost certainly wrong?

Options.

- A — The payload URL is wrong
- B — The secret in GitHub and the secret on your server do not match
- C — The token needs Contents write permission
- D — GitHub has not verified the webhook yet

Answer. B.

If they get it right. Right — and the reason this one is nasty is that both halves look correct in isolation. GitHub shows a secret is set; your server has a secret set. Neither dashboard can tell you they are different strings, so the only symptom is a signature that never matches.

If they get it wrong. The two secrets do not match. The signature check recomputes the HMAC from the secret on your server and compares it to the header GitHub sent — if the strings differ by even one character, or the environment variable was never actually set on the host, every delivery fails the comparison and returns 403. A wrong payload URL gives no delivery at all, not a rejected one; the token's Contents permission governs *committing*, which has nothing to do with inbound verification; and GitHub does not hold webhooks back for verification the way Slack does. When a webhook 403s, check that the env var is really set on the server and really matches what is typed into the dashboard.

Screens to capture or re-create

ID	Screen	Must be visible	Must NOT be visible
G1	The agent's repo, files list	An ordinary repo	Real code, real repo names
G2	Settings → Developer settings → Fine-grained tokens	The Generate new token button	Existing tokens
G3	Repository access	Only select repositories , one repo chosen	Real repo names
G4	Permissions	Contents: Read and write	Other permissions toggled on
G5	The generated token	<code>github_pat_</code> prefix then dots, the Copy control	Any real token characters

ID	Screen	Must be visible	Must NOT be visible
G6	A pull request opened by the agent	PR title, author = the agent, the Files changed tab	Real code
G7	Repo → Settings → Webhooks	The Add webhook button, an empty list	—
G8	Add webhook form, empty	Payload URL, Content type, Secret, event selector	—
G9	Payload URL filled	An invented app URL ending <code>/webhooks/github</code>	A real deployment URL
G10	Content type	<code>application/json</code> selected	—
G11	Secret filled, and the same value on the host	Masked in both places, side by side	Any real secret
G12	Event selector	<code>repository_invitation</code> ticked	—
G13	Editor: the signature check	HMAC-SHA256 over the body, <code>compare_digest</code> , 403 on mismatch	—

Production: re-create in `animation/walkthrough.html` (recommended) or capture with a scrub pass.

Gate check

READY · 26 beats · 13 [ui] · 4 [card] · 8 [info] · 1 [checkpoint] · ~2:30

- **The two jobs stay separate** — signposted at 3 and 12, and restated as two credentials at 25. That separation is the teaching, not a structural nicety.
- **Checkpoint:** beat 19, between two spoken beats (18 → 20), 4 options, both halves authored. The wrong-answer feedback distinguishes a mismatched secret from a wrong URL and from a permissions problem — the three things people try in the wrong order.
- **The repeated pattern lands** at 22–24: the same signature check as Slack, named as one idea rather than a new trick per vendor. This is the line worth remembering from the whole module.
- **Covers \$5 completely:** fine-grained token scoped to one repo with Contents read/write (5–9), commits and PRs (10–11), the webhook with payload URL, content type and secret (14–18), events including `repository_invitation` (20–21), the verification code shape (22).

Build notes

- **No Imagen art, no i2v.** Cursor travel, click pulses, typed text, toggles.
- **Step counter** (`3 / 7`), advancing on 5, 7, 9, 15, 16, 18, 20.
- **Beat 11 is the hinge sentence** of the module's safety story — "the agent proposes, you decide". Hold it; the assessment quotes it.
- Beat 18 and screen G11 must show the secret in **both places at once** — that side-by-side is what makes the checkpoint's answer obvious in hindsight.
- Beat 22 reuses the exact visual treatment of video 4's HMAC beat, so the repetition is seen, not just said.
- The spoken-point beats are plain typographic cards — one sentence, set large, no illustration.

Video 6 — The Body You Already Have — draft

Format: technical how-to walkthrough. No protagonist. The learner's own hosting dashboard, `card` + `ui` beats.

Covers: guide §6, all three steps plus the sleeping-tier warning

Teaches: you do not set up a new host. You already have a live app somewhere — Railway, Render, Vercel, Fly — and that app is the body this module keeps talking about. Three things to do to it.

One move: paste the secrets, give Slack and GitHub your app's address, add a check-in loop.

Spine: you paste your app's URL into the Slack tab you left open in video 4, and it verifies.

Deliberately platform-agnostic. The reference system runs on Railway, so the code shapes are Railway/Python — but every screen is described by what it *does* ("the page where you paste private values"), not by one vendor's layout. A learner on Render or Vercel must not feel this video is for somebody else.

Script (one beat per line · [card] = full-screen card · [ui] = interface screen · [info] = spoken point on a card)

1	CARD	You do not need a new host. You already have one. <i>(title card)</i>
2	INFO	You have a project deployed somewhere — Railway, Render, Vercel, Fly. That is the body.
3	INFO	Three things left to do to it, and only one is code.
4	CARD	One — tell it the secrets. <i>(signpost)</i>
5	UI	Every platform has one page for private values. <i>(H1 — Variables / Environment)</i>
6	UI	Paste in everything you collected: the Notion key, the Slack tokens, the GitHub token. <i>(H2)</i>
7	INFO	Your code reads these by name. They are never typed into the code itself.
8	CARD	Two — give Slack and GitHub your address. <i>(signpost)</i>
9	UI	Copy your app's normal web address — the link you would send a person. <i>(H3)</i>
10	UI	Paste it into Slack's Event Subscriptions box, the tab you left open. <i>(H4)</i>
11	UI	Slack sends its challenge, and the URL verifies. <i>(H5 — the green tick)</i>
12	UI	Paste it into GitHub's webhook payload URL as well. <i>(H6)</i>
13	INFO	That one step is what actually connects everything.
14	INFO	It is you telling Slack and GitHub: when something happens, call this app.
15	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
16	CARD	Three — add a check-in loop. <i>(signpost)</i>
17	UI	A few lines that run every minute and ask: is there anything new for me? <i>(H7 — the scheduler line)</i>
18	INFO	In Python that is one line. In Node, a cron library does the same.
19	INFO	This is what makes it check in on its own, instead of only reacting to you.
20	UI	How to know it worked: open your logs, send a test message, watch it arrive. <i>(H8 — logs)</i>

- 21 **INFO** Within a second or two. If nothing appears, nothing is listening.
- 22 **UI** One thing to watch: some free tiers fall asleep after a quiet period. (H9)
- 23 **INFO** A sleeping app misses the message until it wakes, so expect a delay, or upgrade the tier.
- 24 **CARD** Secrets in · address out · check in every minute. (checklist)
- 25 **INFO** Your turn. Three steps on the host you already have, then send yourself a test message.

The checkpoint (beat 15)

Stem. You are setting up Slack's Event Subscriptions and you paste in `http://localhost:8000/slack/events`. Slack refuses to verify it. Why?

Options.

- A — Slack requires HTTPS, so `https://localhost:8000` would work
- B — Slack is calling your server from the internet and cannot reach an address that only exists on your laptop
- C — The endpoint path must be exactly `/slack/events`
- D — Event Subscriptions must be toggled on before the URL is accepted

Answer. B.

If they get it right. Exactly, and this is the concrete reason a deployment stops being optional the moment you want inbound events. Outbound calls work fine from a laptop — you are calling Slack. Inbound means Slack calls *you*, and `localhost` is a name that only means anything on the machine you are sitting at.

If they get it wrong. `localhost` means "this machine" — and the machine Slack is calling from is Slack's, not yours. Their servers resolve it to themselves, find nothing, and the verification fails. Switching to `https` does not help because the problem is reachability, not the protocol; the path can be anything you like as long as your app serves it; and toggling Event Subscriptions is what produced the URL box in the first place. This is precisely why you can post messages from your laptop all day but cannot receive a single one until the app lives at a public address — and it is why this video comes after the Slack one rather than before it.

Screens to capture or re-create

ID	Screen	Must be visible	Must NOT be visible
H1	The host's variables page	The page where private values are pasted, generically framed	A real project name; anything vendor-specific enough to exclude other platforms
H2	Variables filled	<code>NOTION_API_KEY</code> , <code>NOTION_GOALS_DB_ID</code> , <code>SLACK_BOT_TOKEN</code> , <code>SLACK_SIGNING_SECRET</code> , <code>SLACK_CHANNEL_ID</code> , <code>GITHUB_TOKEN</code> , <code>GITHUB_WEBHOOK_SECRET</code> — names visible, values masked	Any real value
H3	The app's public URL	The ordinary address bar of the deployed app	A real deployment URL
H4	Slack Event Subscriptions, URL pasted	The Request URL box now filled	—

ID	Screen	Must be visible	Must NOT be visible
H5	Slack's verification tick	The green Verified state	—
H6	GitHub webhook payload URL	The same address plus <code>/webhooks/github</code>	—
H7	Editor: the check-in loop	<code>scheduler.add_job(run_session, "interval", seconds=60)</code> and a one-line Node equivalent beside it	—
H8	The host's logs	A test Slack message arriving, timestamped	Real message content
H9	A free-tier sleep notice	Any platform's "instance spins down when idle" wording, generically framed	Vendor-specific branding that implies only one host

Production: re-create in `animation/walkthrough.html` — strongly preferred here, because a real capture would inevitably be one vendor's dashboard and the whole point is that it does not matter which.

Gate check

READY · 25 beats · 9 [ui] · 5 [card] · 10 [info] · 1 [checkpoint] · ~2:20

- **Platform-agnostic throughout.** No beat says "click the Railway button". Every screen is named by what it does. Railway appears once, in beat 2, in a list of four.
- **Checkpoint:** beat 15, between two spoken beats (14 → 16), 4 options, both halves authored. The wrong-answer feedback explains the outbound/inbound asymmetry, which is the actual insight.
- **Covers §6 completely:** the three steps (4–19), the log test (20–21), the sleeping-tier warning (22–23) — and it closes the loop opened in video 4 by filling in the Request URL at beat 10.
- **The hand-off pays off:** beat 10 explicitly returns to "the tab you left open", so the two videos read as one continuous build rather than two separate tasks.

Build notes

- **No Imagen art, no i2v.** Cursor travel, typed text, the verification tick, log lines arriving.
- **Step counter** (`1 / 3`, `2 / 3`, `3 / 3`) — only three steps, so the counter is a reassurance rather than navigation.
- **Beat 11 is the payoff frame** of two videos: the green tick that video 4 could not reach. Give it a beat of silence after.
- H1 and H9 must be drawn **generically** — if a learner on Render sees a Railway dashboard, the video has failed its own premise.
- Beat 17 shows Python and Node **side by side**, three lines total, so neither language feels like the default.
- The spoken-point beats are plain typographic cards — one sentence, set large, no illustration.

Video 7 — The Four Mistakes — draft

Format: technical how-to. No protagonist. Error screens and diagnosis cards.

Covers: guide \$7, all four

Teaches: four mistakes account for almost every hour lost on this build, and each has a tell you can recognise in seconds — before you open any code.

One move: when it fails, identify which of the four it is first.

Runtime: ~2:15 · HTML and voice only.

Placed last on purpose: the learner has now built the thing, so these land as recognition rather than warning. Every error string appears exactly as it arrives.

Script (one beat per line · [card] = full-screen card · [info] = diagram/data card · [ui] = real screen)

1	CARD	Four mistakes. Each has a tell. <i>(title)</i>
2	INFO	Learn the tells and you stop debugging by guesswork. <i>(fourparts, empty)</i>
3	CARD	One — the credential is set, but the thing was never shared. <i>(signpost)</i>
4	UI	Your Notion call returns object not found, on a database you are looking straight at. <i>(M1)</i>
5	INFO	404 · object_not_found <i>(error card, exact string)</i>
6	INFO	The credential is necessary. It is not sufficient. <i>(statement)</i>
7	INFO	Notion needs Connections. Slack needs an invite. <i>(twocard)</i>
8	INFO	Both ask you to say: let this integration see this specific thing. <i>(twocard detail)</i>
9	CARD	Two — the webhook points somewhere the internet cannot reach. <i>(signpost)</i>
10	UI	Slack refuses to verify your Request URL, and the error says nothing useful. <i>(M2)</i>
11	INFO	You gave it localhost. Slack is calling you, from outside. <i>(diagram — their server → your laptop, blocked)</i>
12	INFO	Outbound works from anywhere. Inbound needs a public address. <i>(twocard)</i>
13	CHECKPOINT	<i>(nothing on screen — the player pauses, the LMS asks, gives feedback, resumes)</i>
14	CARD	Three — the secret is set in one place, not both. <i>(signpost)</i>
15	UI	Every delivery comes back 403, and both dashboards look perfectly fine. <i>(M3)</i>
16	INFO	Nothing can show you that two strings differ. Only the signature check knows. <i>(diagram — dashboard vs env var)</i>
17	INFO	Set it in the dashboard AND as the environment variable. Then confirm it really got set. <i>(checks)</i>
18	CARD	Four — a real key ended up in a tracked file. <i>(signpost)</i>
19	UI	You push, and by morning the token is dead. <i>(M4 — a revocation notice)</i>
20	INFO	That is the good outcome. Someone else could have found it first. <i>(statement)</i>

- 21 **INFO** Only dot env — and dot env is gitignored. (*checks — .env · .gitignore · rotate*)
- 22 **INFO** It stays in git history after you fix the file, so rotate the key too. (*checks, third item lit*)
- 23 **INFO** Key without access · localhost · half-set secret · committed key. (*fourparts, complete*)
- 24 **CARD** Name the tell before you open the code. (*closing*)

The checkpoint (beat 13)

Stem. Your agent posts to Slack perfectly, but has never once received a message. Which mistake is this?

Options.

- A — The credential is set but the channel was never shared
- B — The Request URL points somewhere the internet cannot reach
- C — The signing secret is set in only one place
- D — A key was committed and has been revoked

Answer. B.

If they get it right. Exactly — and the tell is the asymmetry. Sending working proves the token and scopes are fine, because you are the one calling Slack. Receiving nothing points at reachability: either the Request URL is `localhost`, or it points at something that is not running.

If they get it wrong. It is the reachability one, and the giveaway is that *sending works*. If the credential were unshared or the bot uninvited, you could not post at all — so the thing that is working rules that out. A half-set signing secret produces requests that arrive and get rejected, which shows up as 403s in your logs, not silence. A revoked key breaks sending too, loudly and immediately. Silence in one direction while the other is healthy nearly always means nothing is listening at a public address: outbound calls out from anywhere, inbound has to be called *at* somewhere.

Screens

ID	Screen	Must be visible	Must NOT be visible
M1	Terminal + Notion side by side	The 404 <code>object_not_found</code> response, and the database visible in the browser behind it	Real database content
M2	Slack Event Subscriptions	The Request URL box with <code>localhost</code> , and the red verification failure	A real workspace
M3	Webhook deliveries list	A column of 403 responses; the dashboard's secret field showing "set"	Any real secret
M4	A token revocation notice	The provider's "this token was disabled" message, generically framed	A real token or repo

Gate check

READY · 24 beats · 4 [ui] · 14 [info] · 5 [card] · 1 [checkpoint] · ~2:15

- `qa-visuals` **skips this automatically** — no character art. `qa-info` + `qa-checkpoint` are the gates.

- **Checkpoint:** beat 13, between two spoken beats (12 → 14), 4 options, both halves authored. The wrong-answer feedback teaches the diagnostic move — *what is working tells you what is not broken* — which is more durable than the answer itself.
- **Covers §7 completely:** all four mistakes with their tells, "necessary but not sufficient" (6), the localhost reason stated as direction-of-call (11–12), the both-places rule (17), and git history outliving the fix (22).
- **Data templates:** fourparts (2, 23 — one card, empty then complete), error card (5), twocard ×3 (7, 8, 12), diagram ×2 (11, 16), checks (17, 21, 22), statement ×2 (6, 20). Five distinct; two statement cards, at the limit.

Build notes

- **Beat 5 is the only exact error string** on screen: mono, red only on the error name, same treatment as every other error card in the module.
- **Beat 11's diagram is the one people remember** — their server, an arrow, your laptop, and the arrow stopping at a wall. It explains localhost better than any sentence can.
- **Beat 23 reuses the fourparts geometry** used throughout the module, now listing mistakes rather than services. The visual rhyme is intentional.
- No Imagen, no segmentation, no kie. Build is: qa-info → qa-checkpoint → TTS → compile → stitch → verify.

Module 14 — Research Brief: connecting an agentic project to Notion and Slack

Scope: what a **beginner** must actually do, in order, for a Notion + Slack integration to go smoothly — and the specific errors they will hit if they skip a step. Six videos.

A. Verified facts to teach (each with source + date)

#	Fact	Source	Date
F1	There are two ways in, not one: a hosted MCP connector (OAuth, no tokens to manage, tool schemas maintained by the vendor) or an API token (your own code, headless, event-driven). MCP is "the right default for nearly everyone" working <i>interactively</i> in a chat/IDE client; the API is required for anything that must run unattended or react to events, because MCP has no event-subscription surface.	scalekit.com — Notion MCP vs Notion API · StackOne	2026
F2	Notion's hosted MCP server is <code>https://mcp.notion.com/mcp</code> , OAuth-only, added in Claude Code with <code>claude mcp add --transport http notion https://mcp.notion.com/mcp</code> then <code>/mcp</code> to authorise.	developers.notion.com — Connect to Notion MCP · Notion blog	2026
F3	Slack ships an official Slack-hosted MCP server at <code>https://mcp.slack.com/mcp</code> , GA 17 Feb 2026 , user-token OAuth, inherits the authenticating user's permissions and requires workspace-admin approval . It replaced Anthropic's reference implementation (archived May 2025).	docs.slack.dev — Connect to Claude · useearly.com	2026-02-17
F4	Creating a Notion integration grants it zero access. You must open each page/database, then <code>...</code> → Connections → add the integration. Parent pages must be shared too — sharing only a child is not enough. Skipping this returns HTTP 404 object_not_found : "The page, database, or block either does not exist or has not been shared with your integration." This is the single most common Notion API error.	makenotion/notion-mcp-server #81 · SFAI Labs — Notion API key setup	2026
F5	Slack bot token (<code>xoxb-</code>) lives under OAuth & Permissions ; posting needs <code>chat:write</code> (plus <code>chat:write.public</code> to post in public channels it has not joined). Every new scope requires reinstalling the app. Without <code>chat:write.public</code> , or in a private channel, posting fails with <code>not_in_channel</code> until you run <code>/invite @yourbot</code> .	docs.slack.dev — Scopes · Prismatic Slack component docs	2026
F6	Notion API version 2025-09-03 split databases from data sources : a database is now a <i>container</i> of one or more data sources. <code>POST /v1/databases/:id/query</code> becomes <code>POST /v1/data_sources/:id/query</code> ; retrieving/updating a schema and creating pages also move to <code>data_source_id</code> . Database IDs and data source IDs are not interchangeable. The required discovery step: call <code>GET /v1/databases/:database_id</code> with <code>Notion-Version: 2025-09-03</code> and read the returned <code>data_sources</code> array, then persist that <code>data_source_id</code> .	developers.notion.com — Upgrade guide 2025-09-03 · Upgrade FAQs	2025-09-03

#	Fact	Source	Date
F7	The <code>Notion-Version</code> header is required and versions are named for their release date; Notion has continued shipping versions through 2026 (2026-02-01, 2026-03-01, 2026-04-01 among them). Teachable rule: pin a version you have tested and read the upgrade guide before bumping it — do not teach "the latest version is X", it moves.	developers.notion.com — Versioning	2026
F8	Notion enforces an average of 3 requests per second per integration (~2,700 per 15 min), on all plans, with no paid tier for more. Handle 429 by reading the <code>Retry-After</code> header (integer seconds) and pausing at least that long; queue and serialise calls rather than bursting.	developers.notion.com — Request limits	2026
F9	Slack allows no more than one message per second per channel (whether via <code>chat.postMessage</code> , an incoming webhook, or anything else); short bursts tolerated. Web API methods sit in tiers (Tier 1 = 1+/min ... Tier 4 = 100+/min) per method, per workspace, per app. Sustained overrun returns HTTP 429 <code>rate_limited</code> with a <code>Retry-After</code> header.	docs.slack.dev — Rate limits	2026
F10	Slack Events API: your endpoint must return HTTP 200 within 3 seconds . If it does not, Slack retries three times over a few minutes — so if you do the real work (write to Notion, post a message) <i>before</i> acknowledging, you get duplicate actions . Fix: acknowledge first, then work; deduplicate on the stable <code>event_id</code> , not on the retry counter.	docs.slack.dev — The Events API · Question Base writeup	2026
F11	Socket Mode delivers events over a persistent WebSocket instead of a public HTTP endpoint — the correct choice for local development, machines behind a firewall, or anywhere you cannot expose a URL.	docs.slack.dev — The Events API	2026
F12	Notion webhooks: on creating a subscription Notion POSTs a one-time <code>verification_token</code> to your URL, which you paste back into the integration's Webhooks tab to activate — and which then doubles as the signing secret . Every delivery carries <code>X-Notion-Signature</code> (HMAC-SHA256 of the body). Lose the token and you must delete and recreate the subscription.	Hookdeck — securing Notion webhooks · developers.notion.com — Webhooks	2026
F13	A token committed to a public GitHub repo is detected by secret scanning (200+ token formats, 100+ providers, including Slack tokens and webhooks) and reported to the provider, who may revoke it immediately — often before the developer sees the alert. Slack also proactively hunts for its own leaked tokens and disables them. Practice: env vars or a secrets manager, least privilege, rotate.	GitHub Changelog — secret scanning validation for Slack · GitGuardian — remediating Slack bot token leaks	2024-04-11 / 2026
F14	Human-in-the-loop means the agent proposes and a human approves before the action happens — "the key word is before". Put the human where an error is expensive, irreversible, legally significant or hard to detect; let the agent read freely, watch it on a dashboard, and stop it before the irreversible thing. Log the proposed action, approver, timestamp, edits and final state.	Velt — AI agent approval layer · Strata — 2026 guide to HITL	2026

B. Strongest analogies found

USE — the hotel key card (for tokens + scopes). "The front desk doesn't give you a master key. They give you a keycard that opens only your room — and only for the length of your stay. Secure. Limited. Revocable." Scopes are which doors the card opens. (Okta, 2019 — the analogy is evergreen.) It carries the

Notion twist perfectly: in Notion you also have to **hand the card to the room** (share the page), or the card opens nothing.

USE — the doorbell vs the mailbox (for events vs polling). "Polling is checking your mailbox every hour for a package. A webhook is the delivery person ringing your doorbell when it arrives." ([Merge, 2026](#).)

AVOID — "the agent joins your team like a new colleague." Misleads on permissions: a colleague can ask someone for access, an integration silently gets a 404 and stops. It also feeds the "it just knows our workspace" misconception (M1 below).

AVOID — plumbing / "wiring it up". Beginners already believe the hard part is code. The hard part is **permissions**, and pipe metaphors hide that.

C. Top learner misconceptions → these become the failure-mode beats

- **M1. "I made the integration / installed the app, so it can see my workspace."** No. Notion access is granted page by page (F4) and Slack posting needs both a scope and channel membership (F5). This is the number-one beginner wall and owns **video 3**.
- **M2. "An ID from the browser URL is the ID the API wants."** Since 2025-09-03 the thing you query is a **data source**, discovered from the database (F6). Owns **video 4**.
- **M3. "My handler can do the work and then reply to Slack."** Three seconds, three retries, three copies of the same action (F10) — plus the bot that replies to its own message and loops. Owns **video 5**.
- **M4. "I should build with the API because that's the real way."** For interactive work the connector is faster and safer; the API earns its keep when the job runs unattended or reacts to events (F1). Owns **video 2**.
- **M5. "Secrets in the code are fine, it's a private repo."** F13. Owns **video 6**.

D. Current names / numbers safe to put on an **info** card

`https://mcp.notion.com/mcp` · `https://mcp.slack.com/mcp` (GA 17 Feb 2026) · `claude mcp add --transport http notion https://mcp.notion.com/mcp` · `xoxb-` bot token · `chat:write`, `chat:write.public` · `/invite @yourbot` · 404 `object_not_found` · `not_in_channel` · `missing_scope` · `Notion-Version: 2025-09-03` · `GET /v1/databases/:id` → `data_sources[]` → `POST /v1/data_sources/:id/query` · Notion **3 requests/second** · Slack **1 message/second/channel** ·

3 seconds to acknowledge, **3** retries, dedupe on `event_id` · 429 + `Retry-After` · `.env` + `.gitignore` · `X-Notion-Signature`

Do not put on a card: "the latest Notion API version is ..." (F7 — it moves). Say *pin your version*.

E. Best teaching technique for this topic

Error-first, contrast pair. For integration work the durable teaching unit is "*here is the exact error message → here is the one setting that causes it*". Beginners do not fail at concepts, they fail at four or five specific 404s and permission errors, and they cannot tell a permission message from a code bug. So each video stages the real error on screen, names it, and hands over the single fix — paired against the working version (before →

after). Secondary technique: **one worked example run in depth** (one Notion table, one Slack channel, one action) rather than a survey of possibilities.

Ordering follows the actual smooth path: *decide what crosses the boundary (1) → pick the door (2) → get permission right (3) → write structured data (4) → speak without spamming (5) → make it safe (6).*

F. Intro video — Railway and GitHub (added 2026-09-16)

For the module opener, which covers all four connections rather than Notion and Slack alone.

#	Fact	Source	Date
F15	An always-on agent is a persistent-state system , not a memory-augmented model: its behaviour depends on state accumulated across earlier interactions, and its correctness depends on governing that state as much as recalling it. Stateless web infrastructure is poorly suited to it — agents need continuous context, persistent sessions and real-time coordination.	arXiv — Always-On Agents: persistent memory, state and governance · arXiv — Infrastructure for the Agentic Web	2026
F16	Agents must be continuously on and listening for work . Production agents take minutes to iterate, so serverless functions with aggressive timeouts do not fit; Railway runs long-lived services that stay up between requests, historically at 99.96% uptime , for roughly \$10–20/month for a multi-service stack plus provider API fees.	Railway — Running agents on Railway · Railway for Agents	2026
F17	Local is right only when local state matters — "you just built an app, haven't pushed it to GitHub yet, and are running out of the directory you're standing in." Everything autonomous belongs hosted.	Railway blog — agent connections	2026
F18	The number that should frighten people: a 2026 study of 33,596 agent-authored pull requests found 61.38% received no recorded human review at all , and 71.58% of the review comments that did exist were written by agents rather than people. Human review does not scale when agents can produce convincing PRs in volume.	arXiv — Why are agentic pull requests merged or rejected? · arXiv — How humans review AI-generated pull requests	2026
F19	AI-authored changes should be treated as artifacts from an untrusted producer that must prove provenance before review — which is exactly what a pull request is for: a reviewable, reversible, attributable unit of change.	DevOps.com — The agent proposes, the pipeline disposes · GitHub Blog — reviewing agent pull requests	2026

The teaching frame these four facts give. Each connection does one job nothing else can do: Slack is where work is **asked for**, Notion is where it is **remembered**, GitHub is where it **lands reviewably**, Railway is where it **keeps running**. Remove any one and a specific, nameable thing breaks — that is the spine of the intro video, and F18 is the beat that stops GitHub sounding like mere storage.